Project 1

Χρήστος Γιαννάκης
1115202400026

Νικόλαος Κακογιάννης
1115202400219

Overview

The goal of this project is to implement a Heap Files (HP) layer for storing, retrieving, and serially traversing Record type records. The implementation is based directly on the provided Block File (libbf.so library), which simulates a 512-byte buffer pool between RAM and disk.

File layout on disk

A heap file is a sequence of BF-level blocks(512 bytes each). Block 0 holds only the metadata(HP_info) and no records. Blocks 1,2,.. hold the recors in instertion order.
Metadata(HP_info, block0): a file type idientifier(so HP_OpenFile can verify it is a heap file), the total record count, the block count and the iterator state(iterator_pos,iterator_slot)

In-Memory state

HP_info is not passed to the API functions, so the HP layer keeps the state of every open file itself. We use a static array openFiles[BF_MAX_OPEN_FILES] of BF_Block*, indexed by the fileDesc and a counter openCount of open files. Entry openFiles[fileDesc] points to block 0, which stays pinned while the file is open.

Functions

1. **HP_CreateFile:** Calls BF_CreateFile and BF_OpenFile, allocates the first block and writes the initial HP_info into it. It then marks the block dirty, unpins, destroys and closes the file.

2. **HP_OpenFile:** Checks that the open-file limit has not been reached, opens the file with BF_OpenFile and fetches block 0 with BF_GetBlock. It reads the metadata and checks the file-type identifier. On mismatch the file is closed and HP_ERROR is returned. On success block 0 is not unpinned, it is stored in openFiles[*fileDesc] and openCount is incremented.

3. **HP_CloseFile:** Retrieves block 0 from the array, marks it dirty so the latest metadata reaches the disk, unpins and destroys it, clears the array entry, calls BF_CloseFile and decrements openCount.

4. **HP_InsertEntry:** Gets the block count and distinguishes three cases:

   (a) Only block 0 exists (blocks_num == 1): block 1 is allocated and the record goes to slot 0.

   (b) Otherwise the last block is fetched and its counter read. If the counter is below 6, the record goes to slot counter.

   (c) If the counter is 6 (block full), the block is unpinned and a new block is allocated at the end of the file; the record goes to slot 0.

   The record is copied with blockSetRecord, the counter is incremented and written back at byte 456 and the block is marked dirty, then unpinned and destroyed. Finally the metadata is updated (record_countand block_count if a new block was created) and block 0 is marked dirty.

5. **HP_PrintAllEntries:** Scans blocks 1 to $blocks_num - 1$ (an empty file prints nothing and returns HP_OK). If value == NULL, every record is printed with printRecord. Otherwise the comparison is per field: id and postcode as integers (*(int*)value), firstname, lastname and city with strcmp. Each block is unpinned before the next is loaded, so only one data block is pinned at any time.

6. **HP_GetEntry:** Fetches block blockid and reads the record at position with blockGetRecord. It returns HP_ERROR when blockid is not a data block (block 0 holds metadata) or when the position is out of range or not below the block's counter. The block is always unpinned and destroyed before returning.

7. HP_GetNextRecordWithID (iterator): The iterator state, the block and slot where the next call resumes (iterator_pos, iterator_slot), is stored in the metadata of block 0. It therefore survives between calls and is independent for each open file. Each call starts from the saved position and scans blocks, using each block's counter as the bound. When it finds a record with id == *value, it returns it in record, saves the position immediately after it and returns HP_OK, so the next call continues from there and the same record is not returned twice. When the end of the file is reached it returns HP_EOF and resets the iterator to (1, 0), so a new search starts from the beginning.

Build and run

From the project root: make , then run ./bin/myheap -i inputs/records.txt.

Driver(myheap.c): The driver validates its arguments, initialises the block layer with BF_Init(LRU) and creates the heap file DBs/myheap.db with HP_CreateFile. It then opens the file with HP_OpenFile and reads the input text file with fscabf, inserting each record with HP_InsertEntry. On an insertion error it closes the file and exits with failure. After loading, the driver demonstrates the API:

1. HP_PrintAllEntries with value = NULL prints every record in insertion order.

2. HP_GetEntry fetches the record at block 1, slot 2 and prints it.

3. the iterator is shown by repeatedly calling HP_GetNextRecordWithId for id=203, an id shared by three records in the sample input, until it returns HP_EOF. Each call prints te record found and its sequence number, which demonstrates that the iterator resumes after previously returned record rather than restarting. Finally the file is closed with HP_CloseFile